ENGINEERING WHITE PAPER

Monitoring Information Drift in Production Predictive AI Systems

A Unified Framework for Evidence Drift, Log Composition Shift, Calibration Monitoring, Operational Timeliness, and Governance

Version 2.0 | March 2026

Author: Neeraj Rohilla

Executive Summary

Predictive AI systems do not fail suddenly — they decay gradually. As the real-world environments they were trained on continue to evolve, the statistical properties of production data **drift** away from the training baseline. Models that were highly accurate at deployment silently become unreliable, producing incorrect predictions, mis-calibrated confidence scores, and lagging alerts — all without triggering any traditional software alarm. Google's work on production ML technical debt and test readiness argues that monitoring and infrastructure discipline are *first-class requirements* for deployed ML systems, not post-launch extras (Sculley et al., 2015; Breck et al., 2017).

This white paper presents a **systematic, mathematically grounded engineering framework** for detecting and responding to this decay before it causes operational harm. It synthesizes two complementary engineering perspectives: a deep mathematical and implementation reference grounded in information-theoretic derivations and an empirically validated code implementation; and a broader operational playbook covering governance, limitations, and domain adaptation. The result is a framework that is both *rigorous enough to implement from scratch* and *practical enough to deploy across an organization.*

The framework is domain-agnostic in design — applicable to telecommunications network fault detection, financial fraud systems, medical diagnostic AI, industrial IoT anomaly detection, cybersecurity operations, customer support automation, and any other field where deployed models must remain trustworthy in changing environments.

Confidential - Oracle Restricted \Employees Only

The core problem addressed is information drift: the divergence between the statistical world a model was trained on and the statistical world it now operates in. This manifests across four dimensions:

- **Feature distribution shift** — the values of input signals change in character, not just magnitude.
- **Concept drift** — the relationship between inputs and outputs changes over time; the labelling rules themselves shift.
- **Log and event composition shift** — the mix of observable system events changes, invalidating pattern-based detectors.
- **Calibration erosion** — model confidence scores no longer reflect true outcome frequencies, undermining automated decisions that rely on probability thresholds.

The framework addresses all four dimensions through three monitoring layers and one cross-cutting principle:

1. **Layer 1 — Evidence Distribution Health:** PSI for coarse screening; KL/JS divergence (with optional KDE) for shape-sensitive detection of multimodal and categorical mix changes; log template frequency monitoring for event composition shifts.
2. **Layer 2 — Predictive Reliability:** Rolling precision, recall, F1 per class; Brier score and Expected Calibration Error (ECE); recall drift alerting; canary comparison.
3. **Layer 3 — Operational Effectiveness:** TPR/FPR for anomaly detectors; Time-to-Detect (TTD) at median, p90, and p99; SLO-based alerting to prevent both missed incidents and alert fatigue.
4. **Cross-cutting — Segment-Aware Monitoring:** Every metric is computed and stored by ontology entity — the structured identifier encoding the deployment component, version, region, and interface. This localizes the drift to the exact unit that caused it, accelerating root cause analysis.

Core Thesis

A predictive model remains trustworthy only when its live evidence distribution remains interpretable, its confidence estimates stay calibrated, and its alerts continue to arrive in time to be operationally useful. A model that is not actively monitored is not deployed — it is abandoned. The cost of silent model degradation is borne operationally: missed incidents, false alarms, and eroding trust.

Abstract

Offline benchmark accuracy is necessary but insufficient for governing predictive AI in production. Real-world systems face concept drift, data-quality erosion, delayed labels, unstructured operational evidence, and persistent uncertainty about whether a model's confidence estimates remain trustworthy (Sculley et al., 2015; Breck et al., 2017; Gama et al., 2014; Guo et al., 2017). Data problems are often invisible, delayed, and downstream-compounding — a phenomenon documented as *data cascades* in high-stakes AI deployments (Sambasivan et al., 2021). This paper presents a generic engineering framework for production

monitoring that can be applied to supervised classifiers, anomaly detectors, and hybrid rule-plus-ML systems.

The proposed approach integrates: (1) evidence-distribution monitoring with PSI and KL/JS divergence — including kernel density estimation for continuous features and rolling z-score alerting for self-calibrating thresholds; (2) log or event-template frequency monitoring with built-in normalization; (3) predictive quality and calibration tracking including Brier score, Expected Calibration Error, and per-class recall drift detection; and (4) operational effectiveness metrics including true-positive rate, false-positive rate, and time-to-detect at multiple percentiles.

The framework is intentionally domain-agnostic, applies wherever decisions depend on changing live evidence streams, and is designed with two audiences in mind: ML engineers who need mathematical depth and implementation reference, and platform/SRE/risk teams who need governance, limitations, and operational guidance. An empirically validated Python implementation is provided, along with a 55-check automated test suite and full derivations of every metric. The paper also provides metric selection guidance, a governing limitations section, and a curated reference bibliography with links.

Keywords: distribution shift, concept drift, data cascades, Population Stability Index, Jensen–Shannon divergence, KL divergence, model calibration, log template mining, anomaly detection, MLOps, model monitoring, predictive AI reliability, governance, segment-aware monitoring

1. Problem Statement: The Silent Decay of Deployed Models

1.1 The Deployment Assumption Failure

Every supervised machine learning model embeds an implicit assumption: that the statistical distribution of future inputs will resemble the distribution of training inputs. This assumption — sometimes called the *i.i.d. assumption* — is foundational to virtually all classical learning theory (Hastie et al., 2009). In practice, it is rarely perfectly satisfied, and in dynamic operational environments, it is routinely violated in material ways.

The gap between training-time and deployment-time data distributions is collectively referred to as **dataset shift** or **distribution shift**. When this shift is severe enough to materially alter model behavior, it constitutes **concept drift** (Widmer & Kubat, 1996). Crucially, neither phenomenon triggers traditional software monitoring: error rates, latency, and memory consumption remain normal even as model predictions silently degrade.

A compounding factor is that data problems are frequently invisible at the moment they are introduced. Sambasivan et al. (2021) document *data cascades* — situations where data-quality issues propagate silently downstream, appearing much later in model behavior degradation rather than as immediate pipeline errors. A schema change, a parser regression, a rollout-induced shift in event mix, or a change in sensor behavior can quietly undermine a model before any human notices the impact.

1.2 Taxonomy of Drift

Distribution shift takes several distinct forms, each with different implications for monitoring strategy:

Drift Type	Formal Characterization	Operational Manifestation
Covariate Shift	P(X) changes; P(Y|X) stable	Input features shift but labelling rules stay the same. Most common after deployments or seasonal changes.
Prior Probability Shift	P(Y) changes; P(X|Y) stable	The prevalence of classes changes — e.g. fraud rate surges after a new attack campaign.
Concept Drift	P(Y|X) changes	The underlying relationship between inputs and outputs changes. Most severe form; requires model retraining.
Data Quality / Schema Drift	Schema or parsing changes	Upstream changes alter how data is generated, parsed, or encoded without a corresponding label change. Common after software rollouts.
Label Drift	P(Y) changes; labelling lags	Ground-truth labels are delayed or inconsistently applied, making performance metrics unreliable even when the model is unchanged.

1.3 Why Monitoring is a First-Class Requirement

Google's work on hidden technical debt in ML systems argues explicitly that monitoring, serving infrastructure, and data dependency management are *first-class engineering requirements*, not afterthoughts (Sculley et al., 2015). The companion ML Test Score rubric (Breck et al., 2017) operationalizes this by defining concrete readiness tests across data, models, infrastructure, and monitoring — treating production observability as a prerequisite for responsible deployment. Systems that stop at validation-set metrics under-detect training-serving skew, silent feature dropout, and changes in live traffic composition.

2. Shortcomings in Existing Monitoring Systems

Despite widespread recognition of the drift problem in the ML research community, most production monitoring systems remain inadequate. The following shortcomings are documented in both the ML systems literature and operational practice.

2.1 Benchmark-Centric Validation Only

Offline evaluation is essential, but it does not prove production robustness. Breck et al. (2017) argue that deployed ML systems need testing and monitoring around data, pipelines, and serving behavior — not just model training. Systems that stop at validation-set metrics,

therefore, under-detect training-serving skew, silent feature dropout, and changes in live traffic composition.

2.2 Reliance on Aggregate Performance Metrics

Most MLOps platforms expose aggregate accuracy or AUC over rolling windows. These metrics have two critical weaknesses: (a) they require labelled ground-truth data at inference time, which is often delayed or unavailable; and (b) they collapse per-class and per-region signal into a single number. A recall of 0.85 may hide the fact that recall for a critical minority class has dropped from 0.90 to 0.50 in a specific geographic region or release cohort.

2.3 No Feature-Level Distribution Monitoring

Most production systems monitor outputs (predictions, scores) but not inputs (features). Input monitoring is essential because feature distribution shift is the *leading indicator* of downstream accuracy degradation — often detectable hours or days before performance metrics move.

2.4 Sensitivity to Unimodal Assumptions

Where feature monitoring does exist, it commonly relies on simple univariate statistics: mean, variance, and quartiles. These are insensitive to multimodal distribution changes — for example, a latency distribution that maintains its mean at 25 ms while developing a second mode at 80 ms due to a new slow-path code path. Jensen–Shannon divergence over the full distribution detects this change; mean/variance statistics do not (Rabanser et al., 2019).

2.5 Over reliance on PSI Alone

PSI is widely used but insufficient on its own. The commonly cited rule-of-thumb thresholds (0.10 / 0.25) originate from credit risk modelling practice (Anderson, 2007) and lack rigorous statistical justification for general ML settings. Population-shift measures do not by themselves quantify the impact on model outcomes. In engineering terms, **PSI is a screening signal, not a complete monitoring strategy**. KL and JS divergence, combined with rolling statistical thresholds, are required to detect subtler shape changes that PSI misses.

2.6 Absence of Log Behavioral Monitoring

Operational logs are frequently the richest runtime evidence available, yet log monitoring is almost universally separated from model monitoring. Log parsing research (He et al., 2017) demonstrates that structured template extraction from raw logs is the foundation for higher-level anomaly detection and diagnosis. When the composition of log events shifts — new error templates, changed frequency of known templates after a software rollout — this often precedes both metric degradation and incident spikes.

2.7 Unchecked Confidence Scores

Confidence values are often consumed as if they were calibrated probabilities even when they are not. Guo et al. (2017) show that modern neural networks can be poorly calibrated and that post-processing methods such as temperature scaling can materially improve calibration. Any system that routes work, suppresses alerts, or automates action based on model probability should therefore monitor confidence quality — not just top-1 accuracy.

2.8 Static Alert Thresholds

When feature monitoring does exist, alert thresholds are typically fixed constants. Fixed thresholds are insensitive to the natural variability of a given feature: one with high natural variability will generate constant false alarms, while a stable one may fail to alert on meaningful shifts. Rolling statistical thresholds (e.g. $\mu + k\sigma$ over a window of recent observations) are more appropriate but rarely implemented.

2.9 No Operational Latency Tracking

Even well-performing models can silently develop increasing *time-to-detect* — the lag between an incident's true onset and the detector's alert. This degradation is invisible to accuracy metrics but directly impacts operational effectiveness. A detector that fires after the incident window has effectively closed can still score well on retrospective metrics yet fail operationally.

2.10 Lack of Segment-Aware Monitoring

Global aggregate metrics obscure localized drift. A PSI of 0.05 globally may mask a PSI of 0.40 in one specific component, version, or region. Without scoping metrics by ontology entity — the logical unit of deployment — drift is difficult to root-cause and remediate efficiently.

IMPACT	Sculley et al. (2015) describe the hidden technical debt of production ML systems, noting that the ongoing maintenance burden — including data dependency management and monitoring — often exceeds the initial development cost by an order of magnitude. Unmonitored drift is the primary driver of this debt.

3. Proposed Solution: A Layered Drift Monitoring Framework

3.1 Design Principles

The framework is designed around six governing principles:

- **Mathematically grounded:** every alert is derived from a well-defined statistical quantity with known properties and interpretable thresholds.
- **Leading-indicator first:** feature and log monitoring surface drift before performance metrics degrade, enabling proactive response.
- **Segment-aware:** all metrics are computed per deployment entity, enabling localized root-cause analysis.
- **Composable:** the three monitoring layers operate independently but are designed to be interpreted jointly for a richer diagnostic signal.
- **Governance-ready:** all metric values are timestamped, versioned, and associated with the model and entity scope, supporting audit trails and health reviews.
- **Limitation-aware:** the framework explicitly acknowledges that thresholds are local heuristics, labels can lag, and monitoring does not guarantee adaptation quality.

3.2 Reference Architecture

The framework is organized into three horizontal layers and one vertical cross-cutting concern:

Component	Description
Data Sources	Features, events, logs, predictions, alerts, outcomes
Baselines	Training set or last known-good operating window
Metric Engine	Drift, calibration, quality, and timeliness calculations
Segmentation Layer	Region, service, cohort, version, tenant, asset, release, interface
Actions	Investigate, recalibrate, retrain, retune, rollback, document

Layer	Description
Layer 1: Input Monitoring	PSI + KL/JS divergence per feature; log template frequency tracking. Detects upstream data changes before they affect outputs.
Layer 2: Output Monitoring	Precision, Recall, F1 per class; Brier score; ECE. Requires labelled ground truth (often delayed). Confirms whether input drift has affected model behavior.
Layer 3: Operational Monitoring	TPR/FPR for anomaly detectors; Time-to-Detect (TTD). Measures real-world operational impact of any degradation.
Cross-cutting: Alert Engine	SLO-configured thresholds per metric type and entity scope. Generates structured alerts with severity, message, and recommended action. All alerts logged with timestamp and model version.

3.3 Ontology Entity Scoping

Rather than monitoring globally, every metric is computed within the scope of a structured **ontology entity key** encoding the deployment context:

```
entity_key = "<Component>_<Version>::<Region>::<Interface>"

# Telecommunications:
"AMF_v2::RegionA::N11"   "UPF_v3::RegionB::N3"

# Healthcare:
"DiagnosticModel_v3::ICU::LabFeatures"

# Financial services:
"FraudDetector_v5::APAC::CardTransactions"

# Industrial IoT:
"PredictiveMaintenance_v2::PlantB::PressureSensors"

# Cybersecurity:
"ThreatClassifier_v4::TenantA::EndpointEvents"
```

This pattern immediately answers the operationally critical question: *which component, in which region, at which version, is causing the drift?* It reduces root-cause analysis time by eliminating manual decomposition of global metrics.

4. Metric Selection and Interpretation Guide

Metric selection should be driven by the question being asked, the availability of labels, and the structure of the evidence. No single metric can answer all three monitoring questions. The following table provides practitioner guidance for each metric in the framework.

Metric	Best Used For	Strength	Caution / Limitation
PSI	Fast screening of binned numeric or categorical drift	Simple, interpretable, low-compute	Binning-sensitive; common thresholds (0.10/0.25) are operational heuristics from credit risk modelling (Anderson, 2007) — not universal statistical laws for general ML settings
KL Divergence	Directional diagnosis of distribution shift	Sensitive to full-shape changes including multimodal shifts	Asymmetric; requires Laplace smoothing; can be infinite on non-overlapping supports (Cover & Thomas, 2006)
JS Divergence	Dashboarding and alerting on distribution differences	Symmetric and bounded [0, ln2]; finite under smoothing	Still requires baseline windows and local validation; does not replace PSI for speed (Lin, 1991)
Rolling Z-Score Threshold	Per-feature adaptive alerting	Self-calibrates to natural variability; prevents false alarms on volatile features	Requires ≥5 stable batches to warm up; assumes approximate stationarity in the reference window
Template Frequency Shift	Logs, event types, reason codes, alarm codes	Captures failure-mode mix changes before KPIs move (He et al., 2021)	Can reflect parser changes rather than real behavior changes — always investigate both
Precision / Recall / F1	Supervised classification quality	localizes class-level degradation; not collapsed into single number	Requires reliable labels; can lag when ground truth arrives with delay
Brier Score	Probability calibration quality	Proper scoring rule; penalizes both miscalibration and irreducible uncertainty (Brier, 1950)	Can lag if labels arrive late; lower is better [0,1]

Metric	Best Used For	Strength	Caution / Limitation
ECE	Reliability of confidence bins	Directly measures gap between predicted probability and observed frequency (Guo et al., 2017)	Sensitive to bin count choice; use consistent bins across evaluation windows
TPR / FPR	Anomaly and alerting system quality	Directly tied to alert burden and misses; links to operational cost (Chandola et al., 2009)	Trade-off is domain-specific; acceptable FPR varies widely by application
Time-to-Detect	Operational usefulness of the entire detection pipeline	Turns model quality into time value; surfaces late-firing detectors	Requires a trustworthy onset definition; onset labelling quality directly affects TTD validity

4.1 The Three Monitoring Questions

Every metric in the framework answers one of three fundamental questions:

Core Question	Core Metrics	What It Reveals	Typical Actions
Is the evidence changing?	PSI, KL divergence, JS divergence, template frequency shift	Input drift, event-mix changes, parser or schema changes, new operating modes	Investigate rollout, schema, logging, feature engineering, baseline refresh
Is the model still reliable?	Precision, Recall, F1, calibration, Brier score, ECE	Predictive degradation, confidence drift, class-specific blind spots	Recalibrate, relabel, retrain, adjust class thresholds, review features
Is the system operationally useful?	TPR, FPR, Time-to-Detect, coverage	Alert fatigue, missed detections, late detections	Tune alert policies, add canaries, update runbooks, compare champion/challenger

4.2 The Composited Decision Surface

The three layers form a diagnostic hierarchy for interpreting alert combinations:

Observed Signal Pattern	Diagnostic Interpretation
PSI spike; Recall stable	Feature distribution changed but model adapted. Monitor closely. Check for schema changes. Retrain not yet required.

Observed Signal Pattern	Diagnostic Interpretation
JS spike; PSI stable	Subtle multimodal or categorical mix change. Likely an early warning. Increase monitoring frequency.
New log templates; no metric change	Code or parser change upstream. Validate template parser. May precede metric degradation by hours.
Recall drop; PSI stable	Concept drift (P(Y|X) changed) or label quality issue. Requires label investigation before retraining.
All three layers fire simultaneously	Material, broad drift. Strong retraining signal. Escalate to model governance review.
TTD increasing; metrics stable	Operational latency growing. Check feature staleness, alert routing, and detection window size.
ECE increasing; Brier stable	Miscalibration on specific probability ranges. Apply Platt scaling or temperature scaling before full retraining.
Labels delayed; performance metrics absent	Use unsupervised drift signals (PSI, JS, templates) as leading indicators until labels arrive.

3. Mathematical Foundations

3.1 Population Stability Index (PSI)

PSI originated in credit risk modeling as a method for detecting shifts in borrower population characteristics over time (Anderson, 2007). It has since been adopted broadly across industries as a fast, interpretable feature drift screen.

For a feature binned into B intervals using training-set quantile edges:

```
PSI = Σ(i=1 to B) [ (p_curr_i - p_base_i) × ln(p_curr_i / p_base_i) ]

Where:
  p_base_i = fraction of training records in bin i
  p_curr_i = fraction of production records in bin i
  B        = number of bins (typically 10-20)
```

PSI is mathematically equivalent to the *symmetric KL divergence* computed over a shared histogram (i.e., KL(P||Q) + KL(Q||P) over the bins). It is *not* equivalent to the Jensen–Shannon divergence, which uses the mixture distribution as the reference.

Standard interpretation thresholds (Anderson, 2007):

PSI Range	Interpretation
PSI < 0.10	Stable. No action required. Feature distribution consistent with training.
0.10 ≤ PSI < 0.25	Moderate shift. Investigate feature source and upstream dependencies.
PSI ≥ 0.25	Material drift. Model likely degraded on this feature. Retrain or replace.

Laplace smoothing (add-ε) is applied to prevent log(0): each bin proportion is clipped to ε = 10^{-6} and renormalized before computing the log ratio.

3.2 Kullback–Leibler (KL) Divergence

Introduced by Kullback and Leibler (1951), the KL divergence measures the information gained when revising beliefs from distribution Q (baseline) to distribution P (current). In the context of drift monitoring, it quantifies the *surprise* a model trained under Q experiences when processing data drawn from P.

```
KL(P || Q) = Σ_x  P(x) × ln[ P(x) / Q(x) ]     (discrete)

KL(P || Q) = ∫ p(x) × ln[ p(x) / q(x) ] dx     (continuous, via KDE)
```

KL divergence is **asymmetric**: KL(P||Q) ≠ KL(Q||P) in general. It is **unbounded**: it diverges to infinity when P assigns positive probability to regions where Q = 0. For this reason it is used diagnostically but not directly for alerting. The information-theoretic interpretation (Cover & Thomas, 2006) is that KL(P||Q) measures the extra bits required to encode samples from P using a code optimized for Q.

3.3 Jensen–Shannon (JS) Divergence

The Jensen–Shannon divergence (Lin, 1991) addresses both limitations of KL by constructing a symmetric, bounded measure via a mixture distribution M = (P + Q) / 2:

```
JS(P || Q) = (1/2) × KL(P || M) + (1/2) × KL(Q || M)

Where M = (P + Q) / 2

Properties:
  - Symmetric:  JS(P||Q) = JS(Q||P)
  - Bounded:    JS ∈ [0, ln2]  (nats)  or  [0, 1] (bits, divide by ln2)
  - Always finite, even when P and Q have non-overlapping support
  - √JS(P||Q) is a proper metric (Jensen-Shannon distance)
```

JS is the **preferred metric for alerting** in this framework. Its bounded range makes rolling statistical thresholds stable; its symmetry means it is independent of which distribution is labeled baseline vs. current.

3.4 Why KL/JS Detects What PSI Misses

PSI discretizes a continuous feature into a fixed number of bins. If a distribution develops a second mode that falls within existing bin boundaries, PSI may not detect it — the bin counts remain similar. KL/JS over a KDE or fine histogram detects the shape change directly. Example:

```
# Training: N11 latency ~ Normal(25ms, 5ms)
# Production: 70% Normal(25ms, 5ms) + 30% Normal(80ms, 10ms)  [new slow path]

# Mean stays near 32ms — a simple mean check may not alert
# PSI with 10 coarse bins may not alert if the 80ms mode falls in the last bin
# JS divergence over KDE at 200 grid points: ALERTS immediately
#   because the probability mass at 70-100ms has changed from ~0% to ~30%
```

3.5 Kernel Density Estimation (KDE)

When computing JS divergence over continuous features, distributions are estimated via **Gaussian KDE** (Parzen, 1962; Rosenblatt, 1956):

```
ĥ(x) = (1 / n×h) × Σ(i=1 to n) K[ (x - x_i) / h ]

K(u) = (1/√(2π)) × exp(-u²/2)   [Gaussian kernel]

Bandwidth (Scott's rule, Scott 1992):
  h = n^(-1/5) × σ^

  Where σ^ = sample standard deviation of training data
  Minimizes asymptotic mean integrated squared error (AMISE)
```

Densities are evaluated at G = 200 fixed grid points spanning [min_train, max_train] and treated as discrete probability vectors for JS computation. **The grid is fixed from training** to ensure fair comparison across batches.

3.6 Rolling Z-Score Alert Threshold

Fixed thresholds for JS are inappropriate because JS baselines differ across features and domains. Instead, the framework uses a **rolling z-score**:

```
z_t = (JS_t - μ_{t-1}) / σ_{t-1}

Where:
  μ_{t-1} = rolling mean of JS over the last W batches
  σ_{t-1} = rolling std dev of JS over the last W batches
  W       = window size (default: 30 batches)

Alert: FIRE when z_t > k  (default k = 3, i.e. 3-sigma rule)
```

This approach is self-calibrating: features with high natural variability require a larger absolute JS shift to alert, while stable features alert on smaller deviations. The 3-sigma threshold corresponds to a false positive rate of approximately 0.27% under normality, consistent with industrial SPC (Statistical Process Control) practice.

3.7 Log Template Frequency Shift

Raw log lines are normalized to templates by replacing variable tokens (IP addresses, UUIDs, timestamps, numeric values) with typed placeholders. Let T = {t_1, ..., t_V} be the vocabulary of observed templates. The frequency distribution is:

```
p_i = count(t_i) / Σ_j count(t_j)    for each template t_i

New (unseen) templates get mass:  p_new = 1 - Σ_{i ∈ T} p_i^curr

JS and PSI are then applied over this discrete distribution vector:
  JS_log  = JS(p^curr || p^base)    [alert on rolling z-score]
  PSI_log = Σ_i (p_i^curr - p_i^base) × ln(p_i^curr / p_i^base)

Top-shifted templates: rank by |Δ_i| = |p_i^curr - p_i^base|
```

The appearance of new templates (p_new > ε) is treated as an unconditional alert, as new templates indicate code changes, new failure modes, or parser breakage regardless of the aggregate JS value.

3.8 Model Calibration Metrics

Brier Score (Brier, 1950) is a proper scoring rule measuring the mean squared error of predicted probabilities:

```
BS = (1/n) × Σ(i=1 to n) (ŝ_i - y_i)²

Where:  ŝ_i = predicted probability of positive class
        y_i = observed binary outcome {0, 1}
Range: [0, 1]. Lower is better. Perfect calibration + resolution → 0.
```

Expected Calibration Error (ECE) measures the average absolute gap between predicted confidence and observed accuracy across B probability bins:

```
ECE = (1/B) × Σ(k=1 to B) |frac_pos_k - mean_pred_k|

Where:
  frac_pos_k  = fraction of positives in bin k (observed frequency)
  mean_pred_k = mean predicted probability in bin k
```

A model predicting 0.8 confidence for events that occur 0.6 of the time has ECE ≈ 0.2. The reliability curve (frac_pos vs mean_pred) provides a visual calibration diagnostic; the ideal is the 45° diagonal (Niculescu-Mizil & Caruana, 2005).

3.9 Precision, Recall, and F1 (Per-Class)

For a multiclass classifier with classes C = {c_1, ..., c_K}, treating each class as a one-vs-rest binary problem:

```
Precision_k = TP_k / (TP_k + FP_k)     [of predicted k, how many were actually k]
Recall_k    = TP_k / (TP_k + FN_k)     [of actual k, how many were predicted k]
F1_k        = 2 × (Precision_k × Recall_k) / (Precision_k + Recall_k)

Recall drift alert:
  FIRE if  Recall_k(t-L) - Recall_k(t)  >  δ_recall
  Default: δ_recall = 0.20, lookback L = 7 batches
```

Tracking per-class, per-entity recall over time is the primary mechanism for detecting **concept drift** — the relationship between features and labels changing. A global accuracy drop is difficult to act on; a recall drop on a specific class (e.g., “UPF throughput regression”) in a specific region (e.g., “RegionB”) directly points to the affected component and failure mode (Davis & Goadrich, 2006).

3.10 Anomaly Detector TPR/FPR

From the binary confusion matrix at decision threshold τ:

```
TPR(τ) = TP / (TP + FN)     [sensitivity: fraction of anomalies caught]
FPR(τ) = FP / (FP + TN)     [1 - specificity: fraction of normals falsely flagged]
```

```
Operating point selection:
  Choose τ to satisfy  FPR ≤ α_SLO  (alert fatigue budget)
  while maximizing TPR

AUC = P(ŝ_+ > ŝ_-)    [probability correct ranking of random positive vs negative
pair]
```

High FPR causes **alert fatigue** — operators stop trusting and responding to alerts. The framework alerts when FPR exceeds a configurable SLO threshold, triggering threshold recalibration or retraining.

3.11 Time-to-Detect (TTD)

```
TTD_i = max(alert_time_i - onset_time_i,  0)    [seconds]

Summary statistics:
  TTD_50 = Median(TTD_i)    [primary SLO metric]
  TTD_90 = 90th percentile  [captures tail incidents]
  TTD_99 = 99th percentile  [worst-case operational impact]

Alert: FIRE if TTD_50 > τ_TTD_SLO  (default: 300 seconds)
```

Increasing TTD is a critical operational signal invisible to accuracy metrics. It means the detector is systematically falling further behind incident onset — either because feature staleness has increased, the model's decision boundary has drifted, or alert routing latency has grown.

3.12 The Composited Decision Surface

The three monitoring layers form a diagnostic hierarchy:

Observed Signal Pattern	Diagnostic Interpretation
PSI spike; Recall stable	Feature distribution changed but model adapted. Monitor closely. Check for schema changes. Retrain may not be immediately required.
JS spike; PSI stable	Subtle multimodal or categorical mix change. Likely an early warning. Increase monitoring frequency.
New log templates; no metric change	Code or parser change upstream. Validate template parser. May precede metric degradation by hours.
Recall drop; PSI stable	Concept drift (P(Y|X) changed) or label quality issue. Requires label investigation before retraining.
All three layers fire simultaneously	Material, broad drift. Strong retraining signal. Escalate to model governance review.
TTD increasing; metrics stable	Operational latency growing. Check feature staleness, alert routing, and detection window size.
ECE increasing; Brier stable	Miscalibration on specific probability ranges. Apply Platt scaling or temperature scaling before full retraining.

4. Reference Implementation: Python Examples

4.1 Installation and Setup

```
# Dependencies
pip install numpy pandas scipy scikit-learn

# Optional: Drain log template miner
pip install drain3

# Project layout
drift_monitor/
  core/
    psi.py           # PSI monitor
    divergence.py    # KL/JS divergence monitor
    log_templates.py # Log template frequency monitor
    performance.py   # Calibration, Precision/Recall, TPR/FPR, TTD
  monitoring/
    orchestrator.py  # Central DriftOrchestrator
    alerting.py      # SLO-based AlertEngine
  reporting/
    report.py        # HealthReporter, canary comparisons
```

4.2 Computing PSI for a Feature

```
import pandas as pd
import numpy as np
from drift_monitor.core.psi import PSIMonitor

# Training data (baseline)
train_df = pd.DataFrame({
    'n11_latency_ms': np.random.normal(25, 5, 5000),
    'sbi_code': np.random.choice(['200','400','500'], 5000, p=[0.90,0.07,0.03])
})

# Production data (post-rollout, drifted)
prod_df = pd.DataFrame({
    'n11_latency_ms': np.concatenate([
        np.random.normal(25, 5, 3500),   # original population
        np.random.normal(80, 10, 1500),  # new slow path mode
    ]),
    'sbi_code': np.random.choice(['200','400','500'], 5000, p=[0.70,0.07,0.23])
})

# Fit baseline and compute PSI
monitor = PSIMonitor(n_bins=10)
monitor.fit(train_df, ['n11_latency_ms', 'sbi_code'],
            entity_key='AMF_v2::RegionA::N11')

results = monitor.compute(prod_df, ['n11_latency_ms', 'sbi_code'],
                          entity_key='AMF_v2::RegionA::N11')

for r in results:
```

```
    print(f'{r.feature}: PSI={r.psi_score:.4f} [{r.drift_level}]')

# Output:
# n11_latency_ms: PSI=0.3812 [material]   <- bimodal shift detected
# sbi_code:       PSI=0.1547 [moderate]   <- 500 errors surged
```

4.3 Computing KL/JS Divergence with KDE

```
from drift_monitor.core.divergence import DivergenceMonitor

# Use KDE for continuous features (sensitive to multimodal changes)
div_monitor = DivergenceMonitor(
    use_kde=True,
    kde_bandwidth='scott',
    alert_sigma=3.0,     # alert when JS > mu + 3*sigma
    rolling_window=30,   # 30-batch rolling history
)

div_monitor.fit(train_df, ['n11_latency_ms'],
                entity_key='AMF_v2::RegionA::N11')

# Seed rolling history with stable batches
for _ in range(15):
    stable = pd.DataFrame({'n11_latency_ms': np.random.normal(25, 5, 500)})
    div_monitor.compute(stable, ['n11_latency_ms'],
                        entity_key='AMF_v2::RegionA::N11')

# Now check drifted production batch
results = div_monitor.compute(prod_df[['n11_latency_ms']], ['n11_latency_ms'],
                              entity_key='AMF_v2::RegionA::N11')

for r in results:
    print(f'{r.feature}: JS={r.js_score:.4f}, KL={r.kl_score:.4f},
           z={r.z_score:.2f}σ, alert={r.alert}')

# Output:
# n11_latency_ms: JS=0.1823, KL=0.4112, z=4.81σ, alert=True
# (JS correctly detects bimodal shift that mean/variance checks miss)
```

4.4 Log Template Monitoring

```
from drift_monitor.core.log_templates import LogTemplateMonitor

baseline_logs = [
    '2024-01-01T00:01:00Z INFO AMF N11 session setup SUPI 001-01-123456789
latency=22ms',
    '2024-01-01T00:01:01Z WARN AMF N11 timeout SUPI 001-01-234567890 retrying',
    '2024-01-01T00:01:02Z ERROR AMF N11 reject SUPI 001-01-345678901
cause=roaming_not_allowed',
] * 500

# Post-rollout logs include new templates
prod_logs = baseline_logs[:300] + [
    '2024-01-15T00:01:00Z CRIT UPF N3 tunnel TEID 0x12345678 DROPPED rate_limit
threshold=100Mbps',
    '2024-01-15T00:01:01Z ERROR NRF discovery failed service=nsmf-pdusession
attempt=3',
```

```
    '2024-01-15T00:01:02Z WARN PFCP modification failed UPF 192.168.1.10
cause=0x72',
] * 400  # new templates dominate

log_mon = LogTemplateMonitor(alert_sigma=3.0)
log_mon.fit(baseline_logs, entity_key='UPF_v3::RegionB::N3')

result = log_mon.compute(prod_logs, entity_key='UPF_v3::RegionB::N3')

print(f'JS={result.js_score:.4f}, PSI={result.psi_score:.4f}')
print(f'New templates ({len(result.new_templates)}): {result.new_templates[:2]}')
print(result.top_shifted.head(3))

# Output:
# JS=0.2914, PSI=0.4851
# New templates (3): ['CRIT UPF N3 tunnel TEID <NUM> DROPPED ...', ...]
```

4.5 Full Orchestrator — End-to-End

```
from drift_monitor import DriftOrchestrator, HealthReporter

orch = DriftOrchestrator(
    slo_config={
        'psi':        {'warning': 0.10, 'critical': 0.20},
        'ttd_seconds':{'warning': 120,  'critical': 300},
        'fpr':        {'warning': 0.08, 'critical': 0.15},
    }
)

# Register entity with training baseline
orch.register_entity(
    entity_key='AMF_v2::RegionA::N11',
    train_features_df=train_df,
    numeric_features=['n11_latency_ms'],
    categorical_features=['sbi_code'],
    train_logs=baseline_logs,
)

# Run monitoring on a production batch
alerts = orch.run(
    entity_key='AMF_v2::RegionA::N11',
    prod_features_df=prod_df,
    prod_logs=prod_logs,
    y_true_clf=y_true,    # ground-truth class labels
    y_pred_clf=y_pred,    # model predictions
    y_true_calib=y_bin,   # binary ground truth
    y_prob_calib=y_prob,  # predicted probabilities
    y_true_anom=anom_y,   # anomaly labels
    y_score_anom=anom_s,  # anomaly scores
    incidents_df=inc_df,  # columns: onset_ts, alert_ts
    window_id='2025-W30', verbose=True,
)

# Print structured health report
reporter = HealthReporter(orch)
reporter.print_summary()

# Export JSON report for governance
report_json = reporter.full_report(as_json=True)
```

```
with open('model_health_2025W30.json', 'w') as f:
    f.write(report_json)
```

4.6 Canary Comparison

```
# Compare baseline vs canary model before full rollout

orch.register_entity('AMF_v2::RegionA::N11', ...)  # baseline
orch.register_entity('AMF_v3::RegionA::N11', ...)  # canary (new version)

# Run identical production traffic through both
orch.run('AMF_v2::RegionA::N11', prod_features_df=prod_df, ...)
orch.run('AMF_v3::RegionA::N11', prod_features_df=prod_df, ...)

# Side-by-side comparison
comparison = reporter.canary_comparison(
    baseline_entity='AMF_v2::RegionA::N11',
    canary_entity='AMF_v3::RegionA::N11',
)
print(comparison)

#           brier_score   ece    tpr    fpr   median_ttd_s
# baseline        0.098  0.031  0.912  0.064          87.3
# canary          0.071  0.019  0.941  0.041          64.1
#
# Decision: canary is better on all metrics. Proceed with rollout.
```

8. Generalization Across Domains

Every element of this framework is domain-agnostic. The following table illustrates how the same monitoring blueprint applies across eight operational domains by changing only the segmentation scheme, feature schema, template source, and operational definition of onset.

Domain	Typical Segments	Predictive Task	Template / Event Signal	TTD Meaning
Telecommunications / Network Ops	service, region, release, NF version, interface	Incident classification, anomaly alerting	Log template, alarm code, SBI error subtype	Incident onset to first actionable alert
Cloud / SRE Operations	service, region, deployment cohort, tenant	Anomaly alerting, SLO breach prediction	Log template, error code, event type	Degradation onset to PagerDuty alert
Healthcare AI	model version, ward, feature modality, site	Diagnostic classification, risk scoring	ICD code frequency, lab event type, alert template	Clinical deterioration onset to alert

Domain	Typical Segments	Predictive Task	Template / Event Signal	TTD Meaning
Financial Services / Fraud	product, geography, channel, scorecard version	Fraud classification, credit risk scoring	Reason code, transaction event type	First suspicious event to case creation
Manufacturing / Industrial IoT	plant, asset, production line, firmware version	Fault classification, predictive maintenance	Controller alarm template, sensor event class	Deviation onset to maintenance notification
Cybersecurity	environment, tenant, detector version, asset group	Malicious event classification, anomaly alerting	Event type, rule signature, log template	Attack onset to analyst-visible alert
Customer Operations	queue, language, product, policy version	Ticket triage, escalation prediction	Interaction reason code, support event template	Customer issue onset to routed intervention
E-commerce / Digital Platforms	region, user segment, product line, release	Recommendation precision, churn prediction	Search event type, browse session template	Customer intent onset to personalisation trigger

The adaptation process for a new domain consists of three steps: (1) define the ontology entity key structure appropriate to the organisational topology; (2) identify the feature signals and log sources that carry behavioural information for the specific system; and (3) configure SLO thresholds based on operational requirements and historical variability. The mathematical infrastructure and alerting architecture transfer without modification.

8.1 Adaptation Pattern

Design Choice	Change This	Keep This	Reason
Feature names	Column lists in MonitorConfig	Drift calculation logic	Lets the same monitor work for any schema
Segments	group_cols / entity_key structure	Per-segment metric storage	Localises failures instead of hiding them globally
Template column	template_col / regex patterns	Frequency-shift method	Works for logs, reason codes, alarms, or event types
Thresholds	Alert rules and SLOs	Metric definitions	Thresholds must be locally validated against incident history

Design Choice	Change This	Keep This	Reason
Task metrics	Classifier vs anomaly outputs	Three-layer monitoring design	The architecture outlives any one model family

9. Governance, Rollout, and Operating Model

A monitoring system becomes useful only when it is tied to ownership, cadence, and action. Following the spirit of the ML Test Score (Breck et al., 2017), monitoring should be designed as part of production readiness rather than added after launch. Each metric should therefore have an owner, a storage location, a refresh cadence, a thresholding policy, and a documented response path.

9.1 Reference Window Policy

- **Use a stable reference window:** training data, last known-good production period, or a canary cohort. Document which window is used and when it was established.
- **Refresh baselines intentionally:** after retraining, major deployments, or confirmed concept drift. Do not allow baselines to drift silently.
- **Maintain baseline version history:** so that metric trends can be correctly attributed to model changes vs. reference window changes.

9.2 Metric Versioning

Version every metric with: **model version**, **feature pipeline version**, **log parser version**, and **baseline identifier**. This makes post-incident review tractable and prevents ambiguity about what changed when a metric shifts.

9.3 Rollout and Champion/Challenger

- **Maintain champion/challenger comparisons:** during rollouts to detect relative degradation before full cutover.
- **Run canary deployments:** with side-by-side metric tables (Brier score, ECE, TPR, FPR, median TTD) before committing to a new model version.
- **Gate rollouts on monitoring SLOs:** a new model version that cannot demonstrate equivalent or better PSI, calibration, and TTD on the canary cohort should not proceed to full deployment.

9.4 Alert Ownership and Response Paths

Alert Type	Owner	Response Action	Response SLA
PSI > critical threshold	Feature engineer / data engineer	Investigate schema, upstream pipeline, or deployment change	Within 4 hours of alert
JS divergence z > 3σ	ML engineer	Check for multimodal distribution shift; correlate with deployment events	Within 4 hours of alert
New log templates detected	Platform / SRE engineer	Validate parser rules; check for software rollout or new failure modes	Within 2 hours of alert
Recall drop > 20% on key class	Data scientist / model owner	Label quality review; consider targeted retraining	Within 24 hours
ECE > critical threshold	ML engineer	Platt scaling or temperature scaling; consider recalibration	Within 24 hours
TTD median > SLO	SRE / ops team	Investigate alert routing, feature staleness, detection window	Within 2 hours of alert
FPR > critical threshold	ML engineer / ops team	Threshold recalibration; check for benign traffic pattern changes	Within 24 hours

9.5 Review Cadence

- **Weekly:** review all active alerts; separate meaningful drift from acceptable environmental change; update runbooks.
- **Per release:** run canary comparison before full rollout; document metric baseline at deployment.
- **Monthly:** review baseline window validity; assess whether rolling thresholds remain appropriate.
- **Per incident:** conduct post-incident metric review; validate that the monitoring stack would have detected the issue earlier with better configuration.

9.6 Metric Storage Requirements

- Store metric history long enough to support rolling baselines, control bands, and post-incident review — minimum 90 days, recommended 12 months.
- Log all metric values with timestamp, model version, feature pipeline version, baseline identifier, and entity scope.
- Maintain a structured alert log (alert ID, severity, metric type, feature, entity, observed value, threshold, message, recommendation, resolution).

10. Limitations and Practical Cautions

This framework improves observability, but it does not remove the need for engineering judgement. Distribution shift does not automatically mean model failure, and performance degradation does not automatically reveal root cause. The following limitations should be understood before operationalizing any metric from this framework.

IMPORTANT	High-quality monitoring does not guarantee adaptation quality. Retraining, recalibration, and rollback policies must still be independently tested. Monitoring surfaces the signal; humans and processes must interpret and act on it.

10.1 Thresholds Are Local Heuristics, Not Universal Laws

The commonly cited PSI thresholds of 0.10 (moderate) and 0.25 (material) originate from credit risk modelling practice (Anderson, 2007) and have been adopted broadly without rigorous statistical justification for general ML settings. Population-shift measures should not be used as standalone decision rules — they are screening signals only. The correct approach is to treat these thresholds as initial operational heuristics and refine them based on historical incident review in the specific deployment context.

10.2 Labels Can Lag

Calibration and quality dashboards (Layer 2) may update significantly later than evidence-drift dashboards (Layer 1). In many operational settings — anomaly detection, incident management, security analytics, industrial monitoring — ground truth may not arrive for hours, days, or weeks. During this window, the framework's unsupervised Layer 1 signals (PSI, JS, template frequency) are the primary leading indicators. Practitioners should not treat absence of Layer 2 degradation as evidence that the model is healthy if Layer 1 signals are active.

10.3 Template Drift Can Mean Parser Change, Not Behavior Change

A shift in log template frequency can reflect: (a) a genuine change in system behavior — new failure modes, new traffic patterns; or (b) a change in the logging infrastructure — parser updates, log schema changes, agent version updates. Both possibilities must be investigated before attributing template drift to model-relevant evidence change. The new template alert (appearance of unseen templates) is particularly susceptible to parser changes and should trigger a structured investigation rather than an immediate retraining decision.

10.4 Drift Does Not Equal Model Failure

A PSI spike or JS z-score alert indicates that today's data is statistically different from the baseline. It does not automatically indicate that the model's predictions have degraded. In some cases, a model trained on diverse data may generalize well despite a feature shift. The correct response to an input drift alert is investigation and correlation with Layer 2 metrics — not immediate retraining. Premature retraining on shifted data without understanding the cause of the shift can make the situation worse.

10.5 Segment-Awareness Has Overhead

Computing and storing metrics at multiple granularities (global + per segment) requires proportionally more storage, compute, and alert-management capacity. Organizations should start with the segments most likely to exhibit independent failure modes (major release versions,

primary geographic regions, customer tiers) and expand coverage incrementally. An excessively fine-grained segmentation scheme will generate alert volume that overwhelms response capacity.

10.6 Rolling Baselines Require Warm-Up

The rolling z-score alerting mechanism ($\mu + k\sigma$) requires a minimum history of stable batches before it produces meaningful z-scores. The framework defaults to 5 batches minimum, but in practice 15–30 batches provide more reliable baselines. During the warm-up period, absolute threshold rules (e.g. JS > 0.10) should be used as a fallback. New entity keys (e.g. a newly deployed service) will have no rolling history and require manual threshold configuration until sufficient history accumulates.

11. Future Recommendations

The literature and operational experience suggest several areas where organizations can mature beyond today's typical model dashboards. These recommendations are compatible with, but not limited to, the framework proposed here.

11.1 Multivariate Drift Detection

The framework currently applies PSI and JS divergence independently to each feature (univariate monitoring). This may miss joint distributional changes — for example, a correlation structure change between latency and session rate that is invisible in the marginals. Future work should incorporate multivariate drift detection via **Maximum Mean Discrepancy (MMD)** (Gretton et al., 2012) or learned embedding comparison. MMD operates in a reproducing kernel Hilbert space and can detect arbitrary distributional differences in high-dimensional feature vectors without binning.

11.2 Automated Remediation Triggers

The current framework generates structured alerts with diagnostic recommendations, but remediation requires manual intervention. A natural extension is a **closed-loop remediation system**: when specific alert patterns fire (e.g. ECE > 0.10 with stable Brier and stable Recall), the system automatically triggers Platt scaling recalibration without requiring full retraining. For concept drift (Recall drop + label shift confirmed), automated retraining pipeline invocation with the drifted entity's data scope is feasible and consistent with modern MLOps practice (Sculley et al., 2015). Alert ownership policies (§9.4) must be clearly defined before automation is introduced.

11.3 Continuous Labelling Pipelines

The performance monitoring layer requires ground-truth labels at inference time. In many operational settings labels arrive with significant delay or are sparse. Future implementations should integrate **weak supervision** and **semi-supervised label estimation** to maintain performance metric coverage even with delayed or incomplete labels. Techniques such as label

propagation over the ontology graph — where confirmed labels on one entity inform estimates for similar entities — are particularly promising.

11.4 Online and Adaptive Monitoring

The current framework operates on batches. For environments with extremely fast-changing distributions — real-time trading, live network traffic — online versions of the drift statistics are required. The **ADWIN (Adaptive Windowing)** algorithm (Bifet & Gavaldà, 2007) and its variants maintain drift detection with a statistically adaptive window size without requiring a fixed batch size. Integrating these into the existing architecture would support streaming deployments.

11.5 Graph-Structured Ontology for Causal Propagation

The entity key currently encodes ontology context as a string. A more powerful approach represents the ontology as a **directed property graph** — with components, interfaces, regions, and versions as nodes, and dependencies as edges. Drift events can then propagate through the graph: an alert on one interface automatically triggers increased monitoring frequency on dependent components. This transforms the monitoring system from a set of independent entity monitors into a **graph-aware causal monitoring system**, substantially accelerating root-cause localization.

11.6 Standardized Drift Report Schema

The industry currently lacks a standardized schema for ML model health reports. We recommend contributing drift metrics to the **Model Card** format (Mitchell et al., 2019) extended with runtime drift fields, or to the **ML Metadata (MLMD)** schema used by TFX and Kubeflow pipelines. Standardization would enable cross-organizational drift benchmarking and tooling interoperability.

11.7 Uncertainty Quantification Integration

Current calibration monitoring (Brier, ECE) treats prediction uncertainty as a scalar probability. More expressive representations — **conformal prediction intervals**, Bayesian predictive distributions, or deep ensemble variance — provide richer signals for drift. When a model's conformal prediction set size grows (wider uncertainty intervals for the same inputs), this is a direct, label-free signal of input distribution change that complements JS divergence monitoring (Murphy, 2022).

11.8 Federated Monitoring

In privacy-sensitive deployments (healthcare, finance), raw production data cannot be centralized for monitoring. A **federated monitoring architecture** would compute drift statistics (PSI, JS, template frequencies) locally at each deployment site and transmit only the aggregated statistics — not raw data — to a central monitoring service. Privacy-preserving aggregation via secure multiparty computation or differential privacy can further protect sensitive feature distributions while maintaining monitoring fidelity.

11.9 Institutionalize Pre- and Post-Launch Checklists

Following the ML Test Score model (Breck et al., 2017), monitoring, rollback, and lineage should be *present before full deployment*, not added reactively. Organizations should maintain pre-launch checklists (baseline established? entity key configured? SLO thresholds set? alert

owners assigned?) and post-launch checklists (first-week metrics reviewed? alert cadence appropriate? canary comparison documented?).

References

All citations refer to verifiable, peer-reviewed publications or well-established reference texts. Sources are grouped by topic. Links verified at time of writing.

Foundational Information Theory

[Kullback & Leibler 1951] Kullback, S. & Leibler, R.A. (1951). On information and sufficiency. Annals of Mathematical Statistics, 22(1), 79–86.
Available: https://www.jstor.org/stable/2236703

[Lin 1991] Lin, J. (1991). Divergence measures based on the Shannon entropy. IEEE Transactions on Information Theory, 37(1), 145–151.
Available: https://ieeexplore.ieee.org/document/61115

[Cover & Thomas 2006] Cover, T.M. & Thomas, J.A. (2006). Elements of Information Theory (2nd ed.). Wiley-Interscience.
Available: https://www.wiley.com/en-us/Elements+of+Information+Theory%2C+2nd+Edition-p-9780471241959

Concept Drift and Distribution Shift

[Gama et al. 2014] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. ACM Computing Surveys, 46(4), Article 44.
Available: https://dl.acm.org/doi/10.1145/2523813

[Lu et al. 2018] Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. IEEE Transactions on Knowledge and Data Engineering, 31(12), 2346–2363.
Available: https://ieeexplore.ieee.org/document/8496795

[Widmer & Kubat 1996] Widmer, G. & Kubat, M. (1996). Learning in the presence of concept drift and hidden contexts. Machine Learning, 23(1), 69–101.
Available: https://link.springer.com/article/10.1007/BF00116900

[Rabanser et al. 2019] Rabanser, S., Günnemann, S., & Lipton, Z.C. (2019). Failing loudly: An empirical study of methods for detecting dataset shift. NeurIPS 32.
Available: https://proceedings.neurips.cc/paper/2019/hash/846c260d715e5b854ffad5f70a516c88-Abstract.html

Model Calibration

[Guo et al. 2017] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K.Q. (2017). On calibration of modern neural networks. Proceedings of ICML, PMLR 70, 1321–1330.
Available: https://proceedings.mlr.press/v70/guo17a.html

[Niculescu-Mizil & Caruana 2005] Niculescu-Mizil, A. & Caruana, R. (2005). Predicting good probabilities with supervised learning. Proceedings of ICML, 625–632.
Available: https://dl.acm.org/doi/10.1145/1102351.1102430

[Brier 1950] Brier, G.W. (1950). Verification of forecasts expressed in terms of probability. Monthly Weather Review, 78(1), 1–3.
Available: https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

Classification and Anomaly Detection

[Davis & Goadrich 2006] Davis, J. & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. Proceedings of ICML, 233–240.
Available: https://dl.acm.org/doi/10.1145/1143844.1143874

[Chandola et al. 2009] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), Article 15.
Available: https://dl.acm.org/doi/10.1145/1541880.1541882

Population Stability Index

[Anderson 2007] Anderson, R. (2007). The Credit Scoring Toolkit. Oxford University Press. (Origin of PSI operational thresholds in credit risk modelling.)
Available: https://global.oup.com/academic/product/the-credit-scoring-toolkit-9780199226405

Log Analysis and Template Mining

[He et al. 2017 (Drain)] He, P., Zhu, J., Zheng, Z., & Lyu, M.R. (2017). Drain: An online log parsing approach with fixed depth tree. IEEE ICWS, 33–40.
Available: https://ieeexplore.ieee.org/document/8029742

[Du et al. 2017 (DeepLog)] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. ACM CCS, 1285–1298.
Available: https://dl.acm.org/doi/10.1145/3133956.3134015

ML Systems and Technical Debt

[Sculley et al. 2015] Sculley, D. et al. (2015). Hidden technical debt in machine learning systems. NeurIPS 28, 2503–2511.
Available:
https://papers.nips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html

[Breck et al. 2017] Breck, E. et al. (2017). The ML Test Score: A rubric for ML production readiness and technical debt reduction. IEEE Big Data.
Available: https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/

[Sambasivan et al. 2021] Sambasivan, N. et al. (2021). Everyone wants to do the model work, not the data work: Data cascades in high-stakes AI. ACM CHI.
Available: https://dl.acm.org/doi/10.1145/3411764.3445518

Density Estimation

[Parzen 1962] Parzen, E. (1962). On estimation of a probability density function and mode. Annals of Mathematical Statistics, 33(3), 1065–1076.
Available: https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-33/issue-3/On-Estimation-of-a-Probability-Density-Function-and-Mode/10.1214/aoms/1177704472.full

[Silverman 1986] Silverman, B.W. (1986). Density Estimation for Statistics and Data Analysis. Chapman & Hall.
Available: https://www.routledge.com/Density-Estimation-for-Statistics-and-Data-Analysis/Silverman/p/book/9780412246203

[Scott 1992] Scott, D.W. (1992). Multivariate Density Estimation: Theory, Practice, and Visualization. Wiley.
Available: https://www.wiley.com/en-us/Multivariate+Density+Estimation%3A+Theory%2C+Practice%2C+and+Visualization%2C+2nd+Edition-p-9780471697558

Reference Textbooks

[Bishop 2006] Bishop, C.M. (2006). Pattern Recognition and Machine Learning. Springer.
Available: https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/

[Hastie et al. 2009] Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer.
Available: https://hastie.su.domains/ElemStatLearn/

[Murphy 2022] Murphy, K.P. (2022). Probabilistic Machine Learning: An Introduction. MIT Press.
Available: https://probml.github.io/pml-book/book1.html